Examen 1 — Tech Art 1, cours 00 à 06 et 06 bis

Périmètre : variables, fonctions, listes, boucle `for`, boucle `while`, conditions (cours 00-06 + 06 bis).

Consignes de rendu

- **Épreuve 1 (QCM)** : réponds directement sur l'application QCM d'**École Directe**. 8 points.
- **Épreuve 2 (lecture et raisonnement)** : écris tes réponses dans un **fichier texte** (`.txt`), en numérotant clairement les questions. 6 points.
- **Épreuve 3 (écriture de code)** : le rendu est **un ou plusieurs fichiers de code** (`main.cpp` pour la piste A ; `ofApp.h / ofApp.cpp` pour la piste B — un dossier par exercice si tu en traites deux). 6 points + 2 points bonus.
- À la fin : rassemble le fichier texte et les fichiers de code dans un **zip nommé avec ton nom** (`NOM-Prenom.zip`), et dépose-le dans le **devoir École Directe** ouvert pour cet examen. Seul le QCM ne va pas dans le zip.

Épreuve 1 — QCM (8 points)

Le QCM (16 questions à 0,5 pt) se fait directement sur l'application QCM d'École Directe. Il ne figure pas dans ce document.

Épreuve 2 — lecture et raisonnement (6 points)

Réponses à rédiger dans un fichier texte (`.txt`), questions numérotées — le fichier part dans le zip final.

2a. Lecture de code (3 points — 1 pt par question)

Sujet. Voici un programme complet. Réponds aux trois questions **sans l'exécuter**.

```cpp
#include <iostream>
#include <vector>
using std::cout;
using std::vector;

int main()
{
    vector<int> notes;
    notes.push_back(12);
    notes.push_back(7);
    notes.push_back(15);
    notes.push_back(9);
    notes.push_back(17);

    int compte { 0 };
    int meilleure { 0 };
    for (int i = 0; i < notes.size(); i = i + 1)
    {
        if (notes[i] >= 10)
        {
            compte = compte + 1;
        }
        if (notes[i] > meilleure)
        {
            meilleure = notes[i];
        }
    }
    cout << compte << "\n";
    cout << meilleure << "\n";
    return 0;
}
```

1. Qu'affiche exactement ce programme ? Il ne s'agit pas de dire "la variable compte et la variable meilleure". Il s'agit de dire à quoi correspondent ces variables.
2. On remplace `notes[i] >= 10` par `notes[i] > 12`. Qu'affiche la première ligne dans la conseole ? Pourquoi ?
3. On remplace les cinq `push_back` par trois : `8`, `3`, `6`. Qu'affiche le programme dans la console, et pourquoi la deuxième ligne affiche-t-elle cette valeur ?

2b. Raisonnement pur (3 points — aucune ligne de code attendue)

Sujet. Une salle de spectacle a **50 places**. Les clients se présentent un par un à la caisse et demandent entre 1 et 4 billets. On veut un programme distributeur de bilets qui :

- demande au spectateur **le nombre de place qu'il veut** ;
- vend des billets **tant qu'il reste des places** ;
- **refuse** une demande qui dépasse les places restantes (le client repart, on passe au suivant) ;
- à la fermeture, affiche le **nombre de clients servis** et le **nombre de billets vendus**.

On ne te demande **pas de code**.

1. **(1,5 pt)** Liste les variables que tu créerais : pour chacune, son **nom**, son **type**, et **à quoi elle sert**.
2. **(1,5 pt)** Décris en français, étape par étape, le déroulement du programme : ce qui se passe, ce qui se répète, dans quel ordre, et quand on s'arrête.

Épreuve 3 — écriture de code (6 points)

Vous choisissez un exercice parmis les quatre suivants et vous l'implémentez en suivant les étapes. Si vous le finissez avant la fin, pour pouvez en choisir un autre pour deux points bonus.

Rendu : un ou plusieurs fichiers de code (`main.cpp` pour la piste A, `ofApp.h` / `ofApp.cpp` pour la piste B), à placer dans le zip final — un dossier par exercice si deux exercices sont traités.

Principe : les **étapes du raisonnement sont données**, vous ecrivez le code. Le sujet propose **quatre exercices** — deux d'informatique de gestion (console, piste A), deux d'application graphique openFrameworks (piste B).

Piste A — informatique de gestion (console)

Exercice A1 — Le relevé de notes

Écris un programme console qui gère un relevé de notes : l'utilisateur saisit 8 notes (notées sur 20). Le programme affiche ensuite la moyenne, le nombre de notes supérieures ou égales à 10, et une appréciation : « Felicitations » si la moyenne atteint 14, « Satisfaisant » si elle atteint 10, « Insuffisant » sinon.

Les étapes à suivre :

1. crée une liste de nombres et les variables de calcul ;
2. une boucle de 8 tours : demande ā l'utilisateur une note (avec std::cin) et range-la dans la liste ;
3. une boucle sur la liste : accumule le total et compte les notes ≥ 10 ;
4. calcule la moyenne — attention à la division : 8 est un entier, la moyenne ne doit pas être un entier ;

5. affiche la moyenne et le compte, puis l'appréciation avec une chaîne de `if` / `else if` / `else`.

Exercice A2 — La paie de la semaine

Écris un programme console de paie : l'utilisateur saisit les heures travaillées pour chacun des 5 jours de la semaine. Le tarif est de 12 euros de l'heure ; chaque heure au-delà de 35 heures hebdomadaires est payée 1,5 fois le tarif. Le programme affiche le total d'heures et le salaire de la semaine.

Les étapes à suivre :

1. crée les variables : le total d'heures, la saisie du jour, le salaire (nombre à virgule) ;
2. une boucle de 5 tours : demande à l'utilisateur les heures pour chacun des cinq jours et accumule ;
3. après la boucle : si le total est inférieur ou égal à 35, le salaire est simplement total × tarif ;
4. sinon : 35 heures au tarif normal, et les heures au-delà au tarif majoré ;
5. affiche le total d'heures et le salaire.

Piste B — application graphique (openFrameworks)

Exercice B1 — La jauge de vie

Dans `ofApp.h` / `ofApp.cpp` (fenêtre 800 × 800), programme une jauge de vie de jeu vidéo : une barre horizontale en haut de l'écran, pleine au départ, qui se vide toute seule au fil du temps. Sa couleur suit son niveau : verte au-dessus de la moitié, orange au-dessus du quart, rouge en dessous. Un clic de souris la recharge complètement.

Les étapes à suivre :

1. dans le `.h` : une variable partagée pour la vie (de 0 à 300, la largeur de la barre en pixels), et une pour la vitesse de perte (en pixels par seconde) ;
2. dans `update()` : fais baisser la vie et empêche-la de passer sous 0 ;
3. dans `draw()` : fond sombre, puis choisis la couleur avec une chaîne de `if` / `else if` / `else` selon le niveau, et dessine un rectangle de largeur égale à la vie ;
4. dans `mousePressed()` : remets la vie au maximum.

Exercice B2 — La pluie

Dans `ofApp.h` / `ofApp.cpp` (fenêtre 800 × 800), programme une pluie : 20 gouttes (des petits cercles) tombent du haut de l'écran à des vitesses différentes. Quand une goutte sort par le bas, elle repart en haut, à une position horizontale tirée au hasard.

Les étapes à suivre :

1. dans le `.h` : trois listes partagées — les x, les y, les vitesses ;
2. dans `setup()` : une boucle de 20 tours qui remplit les trois listes — x au hasard dans la largeur, y au hasard dans la hauteur, vitesse au hasard entre 60 et 180 ;
3. dans `update()` : une boucle sur les listes — chaque y avance de sa vitesse ; si un y dépasse le bas de la fenêtre, remets-le à 0 et tire un nouveau x au hasard ;
4. dans `draw()` : fond sombre, une boucle qui dessine chaque goutte.